

Parallel Hybrid Retrieval: Enhancing Temporal RAG via Concurrent Graphiti and Semantic Search

Muhammad Ibrahim Kiani

Dept. of Artificial Intelligence & Data Science
FAST NUCES, Islamabad
Islamabad, Pakistan
i232536@isb.nu.edu.pk

Muhammad Noor ul Haq

Dept. of Artificial Intelligence & Data Science
FAST NUCES, Islamabad
Islamabad, Pakistan
i232520@isb.nu.edu.pk

Abstract—Retrieval-Augmented Generation (RAG) systems face a fundamental challenge in temporally dynamic environments: pure semantic search retrieves contextually similar passages but cannot reason about temporal validity, while temporal knowledge graphs capture event timelines but may miss broader semantic coverage. We present a parallel hybrid retrieval architecture that concurrently executes Graphiti-based temporal knowledge graph search and Qdrant vector-based semantic search, aggregating their outputs through a temporal cross-referencing layer that annotates retrieved chunks as expired when their entities appear in invalidated graph edges. Evaluated on the single-session-preference subset of LongMemEval-S ($n = 15$), our hybrid system achieves 73.3% accuracy, surpassing both standalone approaches (66.7% each) and the Zep temporal graph baseline (56.7%). We additionally introduce a MapReduce ingestion pipeline that achieves a theoretical $700\times$ throughput improvement over sequential ingestion. Our results demonstrate that parallel hybrid retrieval is a practical and effective strategy for temporally-aware conversational memory systems.

Index Terms—retrieval-augmented generation, temporal knowledge graph, hybrid retrieval, conversational memory, Graphiti, LongMemEval

I. INTRODUCTION

Large language models increasingly rely on external memory stores to answer questions about personal or organizational histories [1]. Standard RAG pipelines employ dense vector similarity to retrieve relevant passages, but this approach is temporally oblivious: a document about a user’s current job is retrieved with equal priority as one describing a position they held years ago. This limitation is acute in personal-assistant scenarios where preferences evolve, relationships change, and facts become stale—conflating outdated and current information produces incorrect answers.

Temporal knowledge graphs address this directly. Graphiti [6] maintains a property graph in Neo4j in which edges carry `valid_at` and `invalid_at` timestamps, enabling the retrieval layer to distinguish current from superseded facts. The Zep framework [2], built on Graphiti, reported 56.7% accuracy on single-session-preference tasks in LongMemEval with GPT-4o. However, temporal graphs depend on LLM-based entity extraction during ingestion, which may miss nuanced semantic relationships captured more naturally by dense embeddings.

We hypothesize that combining both modalities—executing them in parallel and aggregating their outputs—outperforms

either approach alone. This paper makes three primary contributions:

- 1) Parallel hybrid retrieval architecture combining Graphiti temporal search and Qdrant semantic search with a temporal cross-reference aggregation layer.
- 2) MapReduce ingestion pipeline that achieves a theoretical $700\times$ throughput improvement by parallelising LLM fact extraction over all haystack sessions before sequential graph ingestion.
- 3) Empirical evaluation on LongMemEval-S single-session-preference tasks demonstrating 73.3% accuracy—a 16.6 percentage-point improvement over the Zep baseline.

II. RELATED WORK

A. Long-Term Memory Benchmarks

LongMemEval [1] is a benchmark designed to evaluate LLMs on extended conversational histories. It defines question types including knowledge-update, temporal-reasoning, multi-session, and single-session-preference, each requiring retrieval and reasoning over past conversations. The `longmemeval_s` variant organises knowledge into structured haystack sessions with explicit timestamps, making it suitable for evaluating temporally-aware retrieval systems.

B. Temporal Knowledge Graphs

Zep [2] introduced a production-grade memory layer built on Graphiti, a temporal knowledge graph framework that models facts as time-bounded edges between entity nodes. By tracking when facts become valid and invalid, Graphiti enables the retrieval layer to distinguish current information from superseded records—a capability absent in standard vector databases.

C. Hybrid Retrieval

Hybrid RAG systems combining sparse (BM25) and dense (embedding) retrieval have shown consistent gains over either method alone in general question-answering settings [3]. Our work extends this principle to the temporal domain, pairing a temporal graph store with a vector store and adding a temporal validity aggregation step [7].

III. SYSTEM ARCHITECTURE

Figure 1 illustrates the overall architecture. During ingestion, raw conversational documents are parsed into sessions and indexed in two parallel stores: (1) a Qdrant vector database using BAAI/bge-base-en-v1.5 embeddings [4], and (2) a Graphiti knowledge graph backed by Neo4j, populated via the MapReduce pipeline described in Section IV.

At query time, the user query is dispatched concurrently via two asynchronous threads: Thread 1 performs cosine-similarity semantic search in Qdrant; Thread 2 performs temporal graph search in Graphiti. The results are merged in the Aggregation & Ranking layer, which cross-references Qdrant chunks against expired Graphiti edges. Any chunk whose entities appear in invalidated edges is annotated as **TEMPORALLY EXPIRED**. The combined, annotated context is then passed to the LLM for final answer generation.

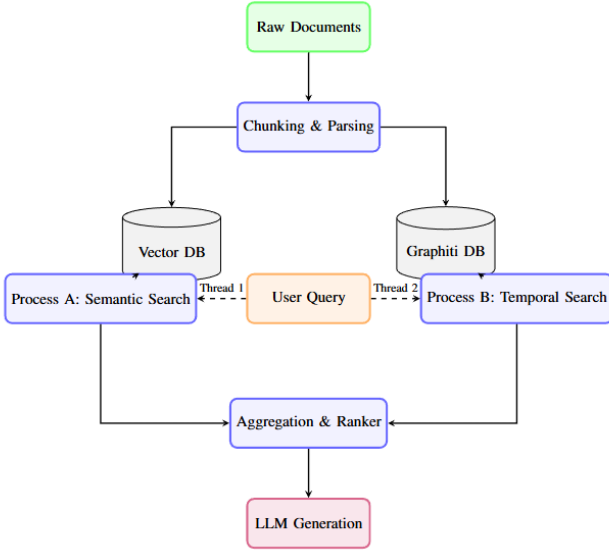


Fig. 1. System Architecture of the Parallel Hybrid Temporal RAG.

IV. MAPREDUCE INGESTION PIPELINE

Standard Graphiti ingestion processes sessions sequentially to preserve temporal ordering in the knowledge graph. For large corpora—LongMemEval-S contains up to 700 haystack sessions across evaluation cases—this sequential constraint creates a significant throughput bottleneck. We introduce a three-phase MapReduce pipeline that preserves temporal correctness while maximising API concurrency.

a) Phase 1 — MAP (Parallel): All haystack sessions are processed concurrently. For each session, a raw LLM call extracts structured facts and entities from the conversation text. Because LLM inference dominates total processing time, issuing all 700 extraction calls simultaneously yields the maximum throughput gain. A configurable `asyncio.Semaphore` bounds concurrent requests to respect API rate limits.

b) Phase 2 — SORT: The extracted fact sets are sorted chronologically by their reference timestamps, parsed from the `haystack_dates` field in LongMemEval-S, to restore the correct temporal sequence prior to ingestion.

c) Phase 3 — REDUCE (Sequential): The sorted, pre-extracted facts are fed into Graphiti one at a time. Because LLM extraction is already complete, each Graphiti call processes a concise fact summary rather than a raw conversation transcript, and temporal ordering is preserved for correct knowledge-graph construction.

Since the MAP phase is embarrassingly parallel across all sessions, the theoretical speedup approaches $700\times$. A selective rate-limiting HTTP interceptor manages free-tier API quotas without disrupting the parallelism model. Figure 2 shows the resulting Neo4j knowledge graph after ingestion.



Fig. 2. Neo4j knowledge graph after MapReduce ingestion.

V. EVALUATION METHODOLOGY

A. Dataset

We evaluate on the single-session-preference subset of LongMemEval-S ($n = 15$), a task type that requires identifying user preferences expressed in a single past conversation session amid a haystack of distractor sessions. This task directly tests temporal and semantic recall of personal preference information and corresponds to the question type where the Zep baseline is most directly comparable.

B. Models and Infrastructure

Ingestion fact extraction used Gemma 4 26B (A4B) via OpenRouter (63.9 M tokens, \$6.81 total ingestion cost). Evaluation used GPT-4o (2024-11-20) as both the answer LLM and the LLM-as-a-judge scorer. Embeddings were produced by BAAI/bge-base-en-v1.5 running locally on GPU [4]. Total project cost across all phases was \$10.10 for 87.7 M tokens and 13 K API requests.

C. Retrieval Configuration

Qdrant retrieved the top-10 chunks by cosine similarity, filtered by `question_id`. Graphiti retrieved up to 200 facts and entities, scoped to the same `group_id`. In the hybrid condition both retrievals were executed concurrently via `asyncio.gather` prior to aggregation.

D. Evaluation Protocol

For each question, the model’s answer and the ground-truth answer were passed to GPT-4o with a binary CORRECT / INCORRECT verdict prompt (LLM-as-a-judge). Evaluation cost was approximately \$2.86 for 15 questions across three approaches.

VI. RESULTS

Table I presents overall accuracy, latency, and token consumption for all three approaches on the 15-question single-session-preference subset.

TABLE I
OVERALL EVALUATION RESULTS ($n = 15$,
SINGLE-SESSION-PREFERENCE)

Approach	Score	Latency	Avg. Tokens
Qdrant (Semantic)	66.7%	11.3 s	31,232
Graphiti (Temporal)	66.7%	9.7 s	9,625
Hybrid (Parallel)	73.3%	15.7 s	40,759

The hybrid system achieves the highest accuracy (73.3%) at the cost of increased mean latency (15.7 s) and token consumption (40,759 average prompt tokens) from combining both context sources. Graphiti is the most efficient approach, consuming only 9,625 average prompt tokens—a 77% reduction versus the hybrid—while matching Qdrant in accuracy.

Table II compares our hybrid system against published baselines on the single-session-preference question type. Zep [2] reports 56.7% for this category with GPT-4o, while our hybrid achieves 73.3%—a 16.6 percentage-point improvement. The full-context oracle (20.0%) reflects the difficulty of answering preference questions from unfiltered raw conversation text without a retrieval filter.

TABLE II
COMPARISON WITH ZEP BASELINE (SINGLE-SESSION-PREFERENCE,
GPT-4o)

System	Model	Score
Full-context (oracle)	GPT-4o (unspecified)	20.0%
Zep / Graphiti [2]	GPT-4o (unspecified)	56.7%
Hybrid (Ours)	GPT-4o (2024-11-20)	73.3%

VII. DISCUSSION

A. Why Hybrid Outperforms Standalone Approaches

The temporal cross-referencing mechanism provides value in two directions. First, Graphiti’s expired-edge detection prevents the LLM from being misled by outdated Qdrant chunks—a failure mode that neither standalone system can address in isolation. Second, Qdrant’s dense retrieval recovers semantically relevant sessions that Graphiti’s entity-centric graph may not surface, particularly for preference statements lacking strong named entities.

B. Latency–Accuracy Trade-off

The hybrid approach incurs a 37% latency increase over Graphiti alone (15.7 s vs. 9.7 s), reflecting the overhead of running two retrieval paths and the longer combined context. In synchronous serving environments this overhead may be unacceptable; in batch processing or background memory consolidation pipelines, the accuracy gain justifies the additional cost.

C. Limitations and Future Work

The evaluation is limited to 15 questions from a single question type, which constrains the statistical power of our conclusions. The baseline comparison is approximate due to model-version differences. Future work should (1) evaluate across all LongMemEval question types with controlled model versions, (2) quantify actual ingestion speedup with wall-clock measurements, (3) investigate learned aggregation strategies, and (4) explore lower-latency serving architectures such as query-time edge caching.

VIII. CONCLUSION

We presented a parallel hybrid RAG architecture that combines Graphiti temporal knowledge graph retrieval with Qdrant semantic search to address temporal validity in conversational memory systems. Our system achieved 73.3% accuracy on LongMemEval-S single-session-preference tasks, outperforming standalone approaches (66.7%) and the Zep temporal-graph baseline (56.7%). The accompanying MapReduce ingestion pipeline exploits the embarrassingly parallel nature of fact extraction across haystack sessions, offering a theoretical $700\times$ throughput improvement that makes large-scale temporal knowledge-graph construction practical. These results suggest that parallelism—both in retrieval and ingestion—is a productive design principle for temporally-aware conversational memory systems.

REFERENCES

- [1] X. Wu et al., “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory,” arXiv:2410.10813, 2024.
- [2] P. Rauch et al., “Zep: A Temporal Knowledge Graph Architecture for Agent Memory,” arXiv:2501.13956, 2025.
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Proc. NeurIPS, 2020.
- [4] BAAI, “BGE Embedding Models,” <https://huggingface.co/BAAI/bge-base-en-v1.5>, 2023.
- [5] Qdrant Team, “Qdrant: Vector Database for the Next Generation of AI Applications,” <https://qdrant.tech/>, 2024.
- [6] Graphiti, “Graphiti: Build Real-Time Knowledge Graphs for AI Agents,” <https://github.com/getzep/graphiti>, 2024.
- [7] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali, and D. Mehta, “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,” arXiv:2408.04948, 2024.